Sorting algorithms

Outline

Introduction

The sorting problem

Insertion Sort

Exchange Sort

Selection Sort

Comparison of the three algorithms

Introduction

Introduction: sorting

- Sorting is one of the most common tasks in computing
- A sorted list of names, of grades or of prices is much easier to **search**, to **compare** and to **read** than an unsorted one

What we do in this chapter

- We state the **sorting problem** precisely
- We introduce the **criteria** used to compare sorting algorithms
- We study three classical algorithms that solve it:
  - **Insertion Sort**
  - **Exchange Sort**
  - **Selection Sort**

How we study the three algorithms

- They are written in the pseudo-code of the course, with arrays indexed from 1
- For each algorithm we show its execution step by step on the same two arrays,

$$[5, 2, 4, 1, 3] \qquad \text{and} \qquad [12, 11, 13, 5, 6],$$

  so that the three methods can be compared directly
- For each algorithm we count the key comparisons and the moves of keys that it performs, and we establish whether it is in-place and whether it is stable

The sorting problem

Introduction: The sorting problem

- In this section we define the sorting problem: input, output, and what "sorted" means
- We fix the only operations a comparison-based algorithm may perform on the keys
- We introduce the criteria that we use to compare sorting algorithms: comparisons, assignments, extra memory and stability
- These criteria are the yardstick for the three algorithms of the chapter

Definition 1: sorting problem

*Given an array $a[1..n]$ of $n \in \mathbb{Z}_{\geq 1}$ natural numbers, the **sorting problem** consists in rearranging the elements of the array in **non-decreasing order**, i.e., so that*

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

*A **rearrangement** (or **permutation**) of the array is an array that contains the same elements, each with the same multiplicity, in a possibly different order. The output of the sorting problem is the rearrangement of the input array that satisfies the condition above.*

Keys and non-decreasing order

- Each element $a[i]$ is called a **key**: it is the value on which the ordering is based, i.e., the value that is compared to decide the final arrangement
- The order is *non-decreasing* rather than *increasing* because the array may contain equal keys, as in $[3, 1, 3]$, whose sorted rearrangement is $[1, 3, 3]$
- The output must be a rearrangement of the input: an array such as $[0, 0, 0, 0, 0]$ satisfies $a[1] \leq a[2] \leq \cdots \leq a[n]$, but it is not the sorted version of $[5, 2, 4, 1, 3]$

Example

Sorting an array

- Input array: $a = [5, 2, 4, 1, 3]$
- Sorted array: $a = [1, 2, 3, 4, 5]$

Before studying specific algorithms,
we fix the operations that they may use
and the criteria by which we compare them.

Comparison-based sorting

Definition 2: comparison-based sorting

*A sorting algorithm is **comparison-based** if it accesses the keys only by comparing them two at a time and by copying them.*

- All the sorting algorithms studied in this chapter (Insertion Sort, Exchange Sort, Selection Sort) belong to this class: they decide how to rearrange the elements solely on the basis of pairwise key comparisons

Operations on the keys

- The only operations permitted on the keys in a comparison-based sorting algorithm are:
  1. **compare two keys**: determine which of $a[i]$ and $a[j]$ is larger (or whether they are equal);
  2. **copy / assign a key**: move or store a key value, e.g., $tmp \leftarrow a[i]$

- Forbidden operations on keys:
  1. arithmetic on keys (addition, multiplication, etc.);
  2. inspecting individual digits or bits of a key;
  3. any operation that exploits the internal representation of the key

Performance metrics (1/2)

- Since comparisons and assignments are the only operations permitted on the keys, we evaluate a comparison-based sorting algorithm by counting:
  - the **number of key comparisons** it performs;
  - the **number of key assignments** (copies) it performs

- The number of assignments matters in practice. Each **swap** of two array elements requires three assignments:

$$tmp \leftarrow a[i]; \qquad a[i] \leftarrow a[j]; \qquad a[j] \leftarrow tmp,$$

  so algorithms that swap less tend to be faster

Performance metrics (2/2)

- Both numbers depend not only on n but also on the initial order of the keys
- For this reason we compute them in the **best case** (the input that minimizes them) and in the **worst case** (the input that maximizes them), among all arrays with n elements
- In the counts we use the sum of the first $n-1$ positive integers:

$$\sum_{k=1}^{n-1} k \;=\; 1+2+\cdots+(n-1) \;=\; \frac{n(n-1)}{2}$$

In-place sorting and memory usage

- Beyond time, we also care about how much **extra memory** an algorithm uses beyond the space already needed to store the n input keys

Definition 3: in-place sorting

*A sorting algorithm is **in-place** if the amount of extra memory it requires is **constant**, i.e., it does not grow with n.*

In-place algorithms

- Equivalently, an in-place sorting algorithm sorts by rearranging the elements *within* the original array, using at most a fixed number of auxiliary variables (e.g., loop counters, a temporary variable for swapping)

- In-place algorithms typically rearrange the input array by shifting or swapping elements, so they do not require extra space for another array to hold the sorted output

Stability

- In many applications the keys are part of larger **records** (e.g., a student identified by a grade and a name), and two different records may have the same key
- We write a record as a pair (*key*, *label*). Only the keys are compared, and each label travels with its key

Definition 4: stable sorting

*A sorting algorithm is **stable** if records with equal keys appear in the output in the same relative order as in the input.*

Example

Stable and non-stable sorting

- Array of records: $a = [(3, A),\ (1, B),\ (3, C),\ (2, D),\ (1, E)]$
- Stable sorting: $a = [(1, B),\ (1, E),\ (2, D),\ (3, A),\ (3, C)]$
  - The records with key 1 keep the order B, E and the records with key 3 keep the order A, C, as in the input
- Non-stable sorting: $a = [(1, E),\ (1, B),\ (2, D),\ (3, C),\ (3, A)]$
  - The keys are sorted, but both pairs of records with equal keys are inverted

Why stability matters

- Stability matters when the records are sorted several times by different keys
- For instance, sorting students first by name and then, with a stable algorithm, by grade, gives the students with the same grade in alphabetical order

How we evaluate a sorting algorithm

A comparison-based sorting algorithm is evaluated by

- the **number of key comparisons** and the **number of key assignments** it performs (in the best and in the worst case)
- the **extra memory** it uses (in-place or not)
- whether it is **stable**

Summary

The sorting problem:

- Rearrange $a[1..n]$ so that $a[1] \leq a[2] \leq \cdots \leq a[n]$; the output is a rearrangement of the input

Comparison-based sorting:

- Keys are only compared two at a time and copied
- Evaluated by key comparisons and key assignments, in the best and worst case; a swap costs three assignments

Memory and stability:

- In-place: constant extra memory
- Stable: equal keys keep their relative order

Insertion Sort

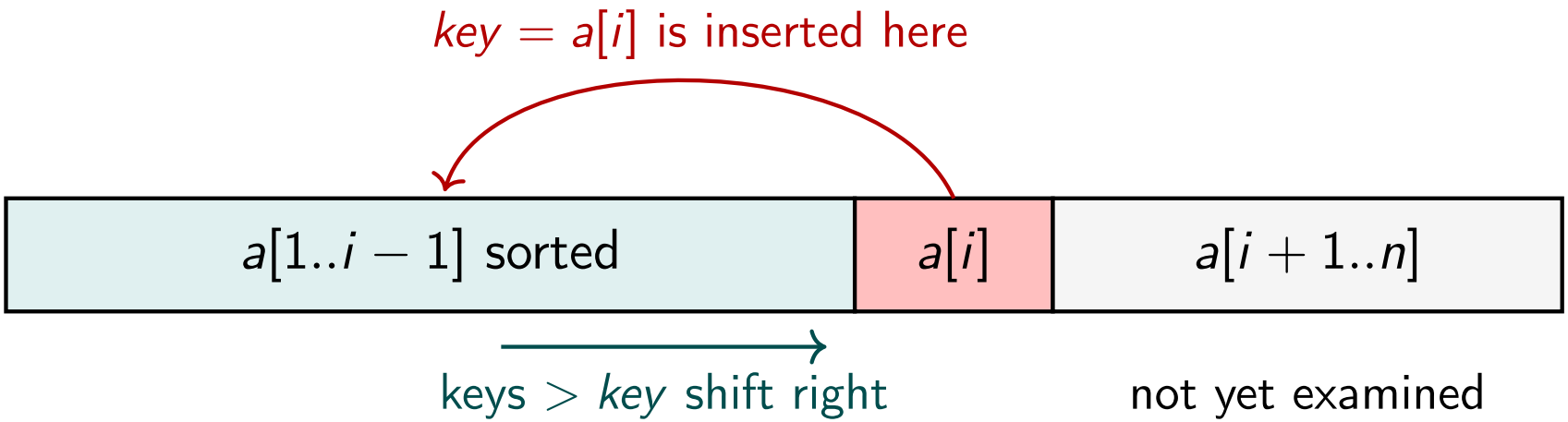
Introduction: Insertion Sort

- The first algorithm: it builds a sorted prefix of the array and inserts each new key in its place
- We give its pseudo-code and run it step by step on the two arrays of the chapter
- We count its comparisons and shifts, and we show that it is in-place and stable
- It is the only algorithm of the chapter that adapts to the input: on a sorted array it does very little work

The idea of Insertion Sort

- Insertion Sort works as many people sort a hand of playing cards: the cards already in the hand are kept sorted, and each new card is inserted in its correct place among them
- At the beginning of iteration i, the sub-array $a[1..i-1]$ is sorted
- The algorithm takes the next element $a[i]$, called *key*, shifts one position to the right all the elements of $a[1..i-1]$ that are greater than *key*, and writes *key* in the position left free

Iteration i of Insertion Sort



- $key = a[i]$ is inserted in the sorted sub-array $a[1..i-1]$, after shifting one position to the right the keys greater than key

Algorithm: InsertionSort (1/2)

Input : An array $a[1..n]$ of n natural numbers
Output : The array a sorted in non-decreasing order

```
1 function InsertionSort(a, n)
2     i := 2
3     while i ≤ n do
4         key := a[i]                          // element to insert
5         j := i − 1
6         while j ≥ 1 and a[j] > key do
7             a[j + 1] ← a[j]                  // shift element to the
                right
8             j ← j − 1
9         end
```

Algorithm: InsertionSort (2/2)

```
function InsertionSort(a, n) (continued)
    while i ≤ n (continued) do
10      a[j + 1] ← key    // insert key in its position
11      i ← i + 1
12  end
13 end
```

Run in Colab: Python · C++ · Java

How InsertionSort works (1/2)

- The outer loop (lines 3–11) iterates through the array starting from the second element ($i = 2$): a single element, $a[1..1]$, is already sorted
- For each element $a[i]$ (the *key*), the inner loop (lines 6–8) scans the sorted portion $a[1..i-1]$ from right to left and shifts one position to the right all the elements greater than *key*

How InsertionSort works (2/2)

- The inner loop stops when $j = 0$ (all the elements of $a[1..i-1]$ were greater than key) or when $a[j] \leq key$
- The condition at line 6 is evaluated from left to right: when $j = 0$ the comparison $a[j] > key$ is not evaluated, since $a[0]$ does not exist
- Line 10 writes key in the position $j + 1$ left free by the shifts
- After each iteration of the outer loop, the sub-array $a[1..i]$ is sorted. When the algorithm stops, $i = n + 1$ and the whole array $a[1..n]$ is sorted

How to read the trace tables

- In the trace tables of this chapter, each row shows the state of the array *after* the operation described in the last column
- The current key is highlighted in red
- The cells modified by a shift, an insertion or a swap are highlighted in blue

Example: Insertion Sort on $[5, 2, 4, 1, 3]$ (1/3)

Input array: $a = [5, 2, 4, 1, 3]$, with $n = 5$. The outer loop executes the iterations $i = 2, 3, 4, 5$.

Detailed trace (inner loop), iterations $i = 2$ and $i = 3$:

i	j	key	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	–	5	2	4	1	3	initial state
2	1	2	5	2	4	1	3	$key \leftarrow a[2] = 2$; $j \leftarrow 1$
	0	2	5	5	4	1	3	$a[1] = 5 > 2$: shift $a[2] \leftarrow 5$; $j \leftarrow 0$
	–	2	2	5	4	1	3	$j = 0$: exit; insert $a[1] \leftarrow 2$
3	2	4	2	5	4	1	3	$key \leftarrow a[3] = 4$; $j \leftarrow 2$
	1	4	2	5	5	1	3	$a[2] = 5 > 4$: shift $a[3] \leftarrow 5$; $j \leftarrow 1$
	–	4	2	4	5	1	3	$a[1] = 2 \leq 4$: exit; insert $a[2] \leftarrow 4$

Example: Insertion Sort on $[5, 2, 4, 1, 3]$ (2/3)

Detailed trace (inner loop), iterations $i = 4$ and $i = 5$:

i	j	*key*	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
4	3	1	2	4	5	1	3	$key \leftarrow a[4] = 1$; $j \leftarrow 3$
	2	1	2	4	5	5	3	$a[3] = 5 > 1$: shift $a[4] \leftarrow 5$; $j \leftarrow 2$
	1	1	2	4	4	5	3	$a[2] = 4 > 1$: shift $a[3] \leftarrow 4$; $j \leftarrow 1$
	0	1	2	2	4	5	3	$a[1] = 2 > 1$: shift $a[2] \leftarrow 2$; $j \leftarrow 0$
	–	1	1	2	4	5	3	$j = 0$: exit; insert $a[1] \leftarrow 1$
5	4	3	1	2	4	5	3	$key \leftarrow a[5] = 3$; $j \leftarrow 4$
	3	3	1	2	4	5	5	$a[4] = 5 > 3$: shift $a[5] \leftarrow 5$; $j \leftarrow 3$
	2	3	1	2	4	4	5	$a[3] = 4 > 3$: shift $a[4] \leftarrow 4$; $j \leftarrow 2$
	–	3	1	2	3	4	5	$a[2] = 2 \leq 3$: exit; insert $a[3] \leftarrow 3$

Example: Insertion Sort on $[5, 2, 4, 1, 3]$ (3/3)

State of the array after each iteration of the outer loop:

i	key	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	5	2	4	1	3	initial state
2	2	2	5	4	1	3	insert 2 at the beginning
3	4	2	4	5	1	3	insert 4 between 2 and 5
4	1	1	2	4	5	3	insert 1 at the beginning
5	3	1	2	3	4	5	insert 3 between 2 and 4

- The sorted array is $[1, 2, 3, 4, 5]$
- The algorithm performs 9 key comparisons and 7 shifts

Example: Insertion Sort on $[12, 11, 13, 5, 6]$ (1/3)

Input array: $a = [12, 11, 13, 5, 6]$, with $n = 5$.

Detailed trace (inner loop), iterations $i = 2$ and $i = 3$:

i	j	key	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	–	12	11	13	5	6	initial state
2	1	11	12	11	13	5	6	$key \leftarrow a[2] = 11$; $j \leftarrow 1$
	0	11	12	12	13	5	6	$a[1] = 12 > 11$: shift $a[2] \leftarrow 12$; $j \leftarrow 0$
	–	11	11	12	13	5	6	$j = 0$: exit; insert $a[1] \leftarrow 11$
3	2	13	11	12	13	5	6	$key \leftarrow a[3] = 13$; $j \leftarrow 2$
	–	13	11	12	13	5	6	$a[2] = 12 \leq 13$: exit; insert $a[3] \leftarrow 13$

Example: Insertion Sort on $[12, 11, 13, 5, 6]$ (2/3)

Detailed trace (inner loop), iterations $i = 4$ and $i = 5$:

i	j	*key*	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
4	3	5	11	12	13	5	6	$key \leftarrow a[4] = 5$; $j \leftarrow 3$
	2	5	11	12	13	13	6	$a[3] = 13 > 5$: shift $a[4] \leftarrow 13$; $j \leftarrow 2$
	1	5	11	12	12	13	6	$a[2] = 12 > 5$: shift $a[3] \leftarrow 12$; $j \leftarrow 1$
	0	5	11	11	12	13	6	$a[1] = 11 > 5$: shift $a[2] \leftarrow 11$; $j \leftarrow 0$
	–	5	5	11	12	13	6	$j = 0$: exit; insert $a[1] \leftarrow 5$
5	4	6	5	11	12	13	6	$key \leftarrow a[5] = 6$; $j \leftarrow 4$
	3	6	5	11	12	13	13	$a[4] = 13 > 6$: shift $a[5] \leftarrow 13$; $j \leftarrow 3$
	2	6	5	11	12	12	13	$a[3] = 12 > 6$: shift $a[4] \leftarrow 12$; $j \leftarrow 2$
	1	6	5	11	11	12	13	$a[2] = 11 > 6$: shift $a[3] \leftarrow 11$; $j \leftarrow 1$
	–	6	5	6	11	12	13	$a[1] = 5 \leq 6$: exit; insert $a[2] \leftarrow 6$

Example: Insertion Sort on $[12, 11, 13, 5, 6]$ (3/3)

State of the array after each iteration of the outer loop:

i	key	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	12	11	13	5	6	initial state
2	11	11	12	13	5	6	insert 11 at the beginning
3	13	11	12	13	5	6	13 already in position
4	5	5	11	12	13	6	insert 5 at the beginning
5	6	5	6	11	12	13	insert 6 between 5 and 11

- The sorted array is $[5, 6, 11, 12, 13]$
- In iteration $i = 3$ the key 13 is already in its position: the inner loop stops at the first comparison and no shift is performed
- The algorithm performs 9 key comparisons and 7 shifts

Properties of Insertion Sort

- The two examples show that the work done by Insertion Sort depends on the initial order of the keys
- A key that is already in its position costs one comparison and no shift
- A key smaller than all the previous ones is shifted across the whole sorted portion

Observation 1: comparisons and shifts of Insertion Sort

Given an array $a[1..n]$ of n natural numbers, `InsertionSort` performs at least $n-1$ and at most $\frac{n(n-1)}{2}$ key comparisons, and at most $\frac{n(n-1)}{2}$ shifts. The lower bounds are attained when the array is already sorted (no shift is performed), the upper bounds when it is sorted in decreasing order with distinct keys.

Proof (1/3)

- Consider iteration i of the outer loop, with $i \in \{2, 3, \ldots, n\}$, and let s_i and c_i be the numbers of shifts and of key comparisons it performs
- The inner loop starts with $j = i - 1 \geq 1$, so $c_i \geq 1$
- Each shift (lines 7–8) follows a comparison $a[j] > key$ that is true, and decreases j by 1; hence $s_i \leq i - 1$

Proof (2/3)

The inner loop then stops in one of two ways:

- with $j = 0$, after $s_i = i - 1$ shifts and without a further comparison: $c_i = s_i = i - 1$;
- after a comparison $a[j] > key$ that is false, with $j \geq 1$: then $s_i = i - 1 - j \leq i - 2$ and $c_i = s_i + 1 \leq i - 1$

In both cases $1 \leq c_i \leq i - 1$ and $0 \leq s_i \leq i - 1$. Summing over $i \in \{2, 3, \ldots, n\}$ and using the sum of the first $n - 1$ positive integers:

$$n - 1 \leq \text{key comparisons} \leq \sum_{i=2}^{n} (i - 1) = \frac{n(n-1)}{2},$$

$$\text{shifts} \leq \frac{n(n-1)}{2}$$

Proof (3/3)

- If the array is sorted, at every iteration $a[i-1] \leq key$: the inner loop stops at the first comparison, with no shift, and the algorithm performs exactly $n-1$ comparisons
- If the keys are distinct and in decreasing order, $key = a[i]$ is smaller than all the keys of $a[1..i-1]$: all of them are shifted and the loop stops with $j = 0$
- So iteration i performs $i-1$ comparisons and $i-1$ shifts, and the totals are both $\frac{n(n-1)}{2}$

□

Assignments of Insertion Sort

- Every iteration also performs two assignments of keys (lines 4 and 10), so the number of key assignments is $2(n-1)$ plus the number of shifts: between

$$2(n-1) \qquad \text{and} \qquad 2(n-1) + \frac{n(n-1)}{2}$$

- Insertion Sort never swaps two elements: a shift is a single assignment, which makes the method cheaper than a sequence of swaps that would move *key* one position at a time

Observation 2: Insertion Sort is in-place and stable

`InsertionSort` is an in-place and stable sorting algorithm.

Proof

- Besides the array, the algorithm uses only the three variables i, j and key, independently of n: it is in-place
- A key is moved to the left only past keys that are *strictly* greater than it (condition $a[j] > key$ at line 6)
- The inner loop stops at the first key $a[j] \leq key$, so key is inserted after every equal key that precedes it
- Hence two records with equal keys are never exchanged, and their relative order in the output is the same as in the input: the algorithm is stable

□

From pseudo-code to code

- In the implementations (C++, Python and Java, in the Colab notebooks above) the array has indices from 0 to $n - 1$
- The outer loop starts from $i = 1$, and the inner loop stops when j becomes -1
- In C++ the operator `&&` evaluates its right operand only when the left one is true, so `a[j]` is never read with $j = -1$

Summary

Insertion Sort:

- Keeps $a[1..i-1]$ sorted and inserts $key = a[i]$ after shifting right the keys greater than it

Cost:

- Between $n-1$ (sorted array) and $\frac{n(n-1)}{2}$ (decreasing array) key comparisons
- At most $\frac{n(n-1)}{2}$ shifts, one assignment each; no swaps

Properties:

- In-place and stable

Exchange Sort

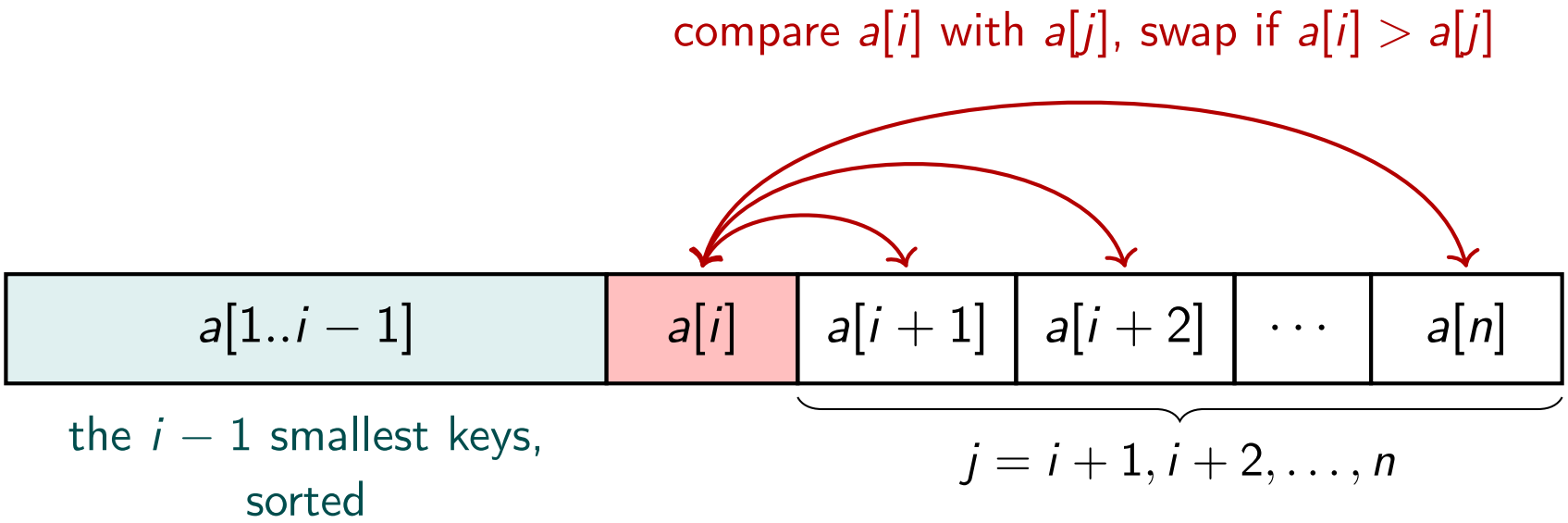
Introduction: Exchange Sort

- The second algorithm follows the opposite idea: it fills the positions one at a time, each with the smallest of the remaining keys
- We give its pseudo-code and run it on the same two arrays
- We will see that it always performs the same number of comparisons, but it may swap many times, and that it is not stable

The idea of Exchange Sort

- Insertion Sort builds the sorted prefix by inserting each new key among the previous ones
- Exchange Sort follows the opposite idea: it fills the positions $1, 2, \ldots, n-1$ one at a time, each with the smallest of the remaining keys
- In iteration i, the algorithm compares $a[i]$ with every element $a[j]$ with $j > i$ and swaps the two elements whenever $a[i] > a[j]$
- At the end of the iteration $a[i]$ is the minimum of $a[i..n]$

Iteration i of Exchange Sort



- $a[i]$ is compared with every $a[j]$, $j > i$, and the two elements are swapped whenever $a[i] > a[j]$; at the end, $a[i] = \min(a[i..n])$

Algorithm: ExchangeSort (1/2)

Input : An array $a[1..n]$ of n natural numbers
Output : The array a sorted in non-decreasing order

```
1  function ExchangeSort(a, n)
2      i := 1
3      while i ≤ n − 1 do
4          j := i + 1
5          while j ≤ n do
6              if a[i] > a[j] then
7                  tmp := a[i]              // swap a[i] and a[j]
8                  a[i] ← a[j]
9                  a[j] ← tmp
10             end
11             j ← j + 1
12         end
```

Algorithm: ExchangeSort (2/2)

```
function ExchangeSort(a, n) (continued)
    while i ≤ n − 1 (continued) do
13      i ← i + 1
14  end
15 end
```

Run in Colab: Python · C++ · Java

How ExchangeSort works

- The outer loop iterates with index i from 1 to $n - 1$
- For each i, the inner loop compares $a[i]$ with every element $a[j]$ for $j \in \{i + 1, i + 2, \ldots, n\}$
- Whenever $a[i] > a[j]$, the two elements are swapped (lines 7–9), so $a[i]$ always holds the smallest key met so far in $a[i..n]$
- After iteration i of the outer loop, $a[i]$ is the minimum of $a[i..n]$ and the sub-array $a[1..i]$ contains the i smallest keys of the array, sorted
- After iteration $n - 1$, the last element $a[n]$ is the largest key, and the whole array is sorted

Example: Exchange Sort on $[5, 2, 4, 1, 3]$ (1/2)

Input array: $a = [5, 2, 4, 1, 3]$, with $n = 5$. The outer loop executes the iterations $i = 1, 2, 3, 4$. Detailed trace (inner loop):

i	j	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	5	2	4	1	3	initial state
1	2	2	5	4	1	3	$a[1] = 5 > a[2] = 2$: swap $a[1] \leftrightarrow a[2]$
	3	2	5	4	1	3	$a[1] = 2 \leq a[3] = 4$: no swap
	4	1	5	4	2	3	$a[1] = 2 > a[4] = 1$: swap $a[1] \leftrightarrow a[4]$
	5	1	5	4	2	3	$a[1] = 1 \leq a[5] = 3$: no swap
2	3	1	4	5	2	3	$a[2] = 5 > a[3] = 4$: swap $a[2] \leftrightarrow a[3]$
	4	1	2	5	4	3	$a[2] = 4 > a[4] = 2$: swap $a[2] \leftrightarrow a[4]$
	5	1	2	5	4	3	$a[2] = 2 \leq a[5] = 3$: no swap
3	4	1	2	4	5	3	$a[3] = 5 > a[4] = 4$: swap $a[3] \leftrightarrow a[4]$
	5	1	2	3	5	4	$a[3] = 4 > a[5] = 3$: swap $a[3] \leftrightarrow a[5]$
4	5	1	2	3	4	5	$a[4] = 5 > a[5] = 4$: swap $a[4] \leftrightarrow a[5]$

Example: Exchange Sort on $[5, 2, 4, 1, 3]$ (2/2)

State of the array after each iteration of the outer loop:

i	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	5	2	4	1	3	initial state
1	1	5	4	2	3	$a[1] = \min(a[1..5]) = 1$ (2 swaps)
2	1	2	5	4	3	$a[2] = \min(a[2..5]) = 2$ (2 swaps)
3	1	2	3	5	4	$a[3] = \min(a[3..5]) = 3$ (2 swaps)
4	1	2	3	4	5	$a[4] = \min(a[4..5]) = 4$ (1 swap)

- The sorted array is $[1, 2, 3, 4, 5]$
- The algorithm performs 10 key comparisons and 7 swaps

Example: Exchange Sort on $[12, 11, 13, 5, 6]$ (1/2)

Input array: $a = [12, 11, 13, 5, 6]$, with $n = 5$. Detailed trace (inner loop):

i	j	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	12	11	13	5	6	initial state
1	2	11	12	13	5	6	$a[1] = 12 > a[2] = 11$: swap $a[1] \leftrightarrow a[2]$
	3	11	12	13	5	6	$a[1] = 11 \leq a[3] = 13$: no swap
	4	5	12	13	11	6	$a[1] = 11 > a[4] = 5$: swap $a[1] \leftrightarrow a[4]$
	5	5	12	13	11	6	$a[1] = 5 \leq a[5] = 6$: no swap
2	3	5	12	13	11	6	$a[2] = 12 \leq a[3] = 13$: no swap
	4	5	11	13	12	6	$a[2] = 12 > a[4] = 11$: swap $a[2] \leftrightarrow a[4]$
	5	5	6	13	12	11	$a[2] = 11 > a[5] = 6$: swap $a[2] \leftrightarrow a[5]$
3	4	5	6	12	13	11	$a[3] = 13 > a[4] = 12$: swap $a[3] \leftrightarrow a[4]$
	5	5	6	11	13	12	$a[3] = 12 > a[5] = 11$: swap $a[3] \leftrightarrow a[5]$
4	5	5	6	11	12	13	$a[4] = 13 > a[5] = 12$: swap $a[4] \leftrightarrow a[5]$

Example: Exchange Sort on [12, 11, 13, 5, 6] (2/2)

State of the array after each iteration of the outer loop:

i	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	12	11	13	5	6	initial state
1	5	12	13	11	6	$a[1] = \min(a[1..5]) = 5$ (2 swaps)
2	5	6	13	12	11	$a[2] = \min(a[2..5]) = 6$ (2 swaps)
3	5	6	11	13	12	$a[3] = \min(a[3..5]) = 11$ (2 swaps)
4	5	6	11	12	13	$a[4] = \min(a[4..5]) = 12$ (1 swap)

- The sorted array is $[5, 6, 11, 12, 13]$
- The algorithm performs 10 key comparisons and 7 swaps

Properties of Exchange Sort

- Unlike Insertion Sort, Exchange Sort compares every pair of positions $i < j$ exactly once, whatever the initial order of the keys
- Only the number of swaps depends on the input

Observation 3: comparisons and swaps of Exchange Sort

Given an array $a[1..n]$ of n natural numbers, `ExchangeSort` performs exactly $\frac{n(n-1)}{2}$ key comparisons and at most $\frac{n(n-1)}{2}$ swaps. No swap is performed when the array is already sorted, and $\frac{n(n-1)}{2}$ swaps are performed when it is sorted in decreasing order with distinct keys.

Proof (1/3)

- In iteration i of the outer loop, $i \in \{1, 2, \ldots, n-1\}$, the inner loop executes line 6 once for each $j \in \{i+1, i+2, \ldots, n\}$, i.e., $n - i$ times
- By the sum of the first $n-1$ positive integers, the total number of key comparisons is

$$\sum_{i=1}^{n-1}(n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

- Each comparison causes at most one swap, so there are at most $\frac{n(n-1)}{2}$ swaps
- If the array is sorted, $a[i] \leq a[j]$ for every $i < j$: the first comparison causes no swap, the array does not change, and the same holds for all the following comparisons

Proof (2/3)

- If the keys are distinct and in decreasing order, let $b[1..n]$ be the input array, so $b[1] > b[2] > \cdots > b[n]$
- In iteration 1, before the comparison with $j \in \{2, 3, \ldots, n\}$ the current value of $a[1]$ is $b[j-1]$ (the last key swapped into position 1), which is greater than $a[j] = b[j]$
- So every comparison causes a swap, which moves $b[j-1]$ to position j

Proof (3/3)

- At the end of iteration 1,

$$a[1] = b[n] \qquad \text{and} \qquad a[2..n] = [b[1], b[2], \ldots, b[n-1]],$$

  which is again in decreasing order

- For instance, $[6, 5, 4, 3, 2, 1]$ becomes $[1, 6, 5, 4, 3, 2]$

- Repeating the argument on $a[i..n]$ for $i = 2, 3, \ldots, n-1$, all the $\frac{n(n-1)}{2}$ comparisons cause a swap

□

Swaps cost more than shifts

- On an array in decreasing order with distinct keys, Exchange Sort performs $\frac{n(n-1)}{2}$ swaps, i.e.,

$$\frac{3n(n-1)}{2} \quad \text{key assignments}$$

- Insertion Sort moves the keys with $\frac{n(n-1)}{2}$ shifts, one assignment each

Observation 4: Exchange Sort is in-place but not stable

`ExchangeSort` is an in-place sorting algorithm, but it is not stable.

Proof (1/2)

- Besides the array, the algorithm uses only the three variables i, j and tmp: it is in-place
- To show that it is not stable, it is enough to exhibit one input on which the relative order of two equal keys changes
- Consider the array of records $[(2, A),\ (2, B),\ (1, C)]$

Proof (2/2)

- For $i = 1$:
  - the comparison with $j = 2$ finds $2 > 2$ false, so no swap;
  - the comparison with $j = 3$ finds $2 > 1$, so $a[1]$ and $a[3]$ are swapped, giving $[(1, C),\ (2, B),\ (2, A)]$
- For $i = 2$: $2 > 2$ is false, so no swap
- The output is $[(1, C),\ (2, B),\ (2, A)]$: the record $(2, A)$, which preceded $(2, B)$ in the input, now follows it

□

From pseudo-code to code

- In the implementations (C++, Python and Java, in the Colab notebooks above) the indices go from 0 to $n-1$
- The outer loop runs with i from 0 to $n-2$, and the inner loop with j from $i+1$ to $n-1$

Summary

Exchange Sort:

- In iteration i, compares $a[i]$ with every $a[j]$, $j > i$, and swaps whenever $a[i] > a[j]$: at the end $a[i] = \min(a[i..n])$

Cost:

- Always exactly $\frac{n(n-1)}{2}$ key comparisons
- Between 0 (sorted array) and $\frac{n(n-1)}{2}$ (decreasing array) swaps, three assignments each

Properties:

- In-place, but not stable

Selection Sort

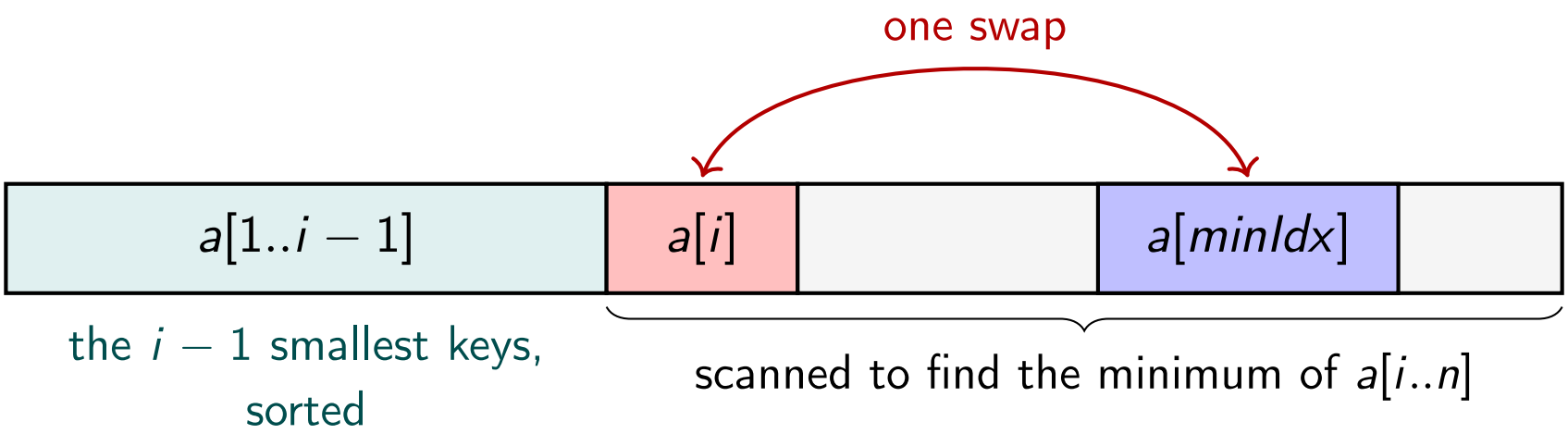
Introduction: Selection Sort

- The third algorithm reaches the same result as Exchange Sort with far fewer swaps
- We give its pseudo-code and run it on the same two arrays
- We will see that it performs at most $n - 1$ swaps, the fewest key assignments in the worst case among the three algorithms, and that it is not stable

The idea of Selection Sort

- Exchange Sort may swap many times in the same iteration only to bring the minimum of $a[i..n]$ into position i
- Selection Sort reaches the same result with at most one swap per iteration:
  - it first *selects* the position of the minimum of $a[i..n]$,
  - and then swaps it with $a[i]$

Iteration i of Selection Sort



- The inner loop finds the position $minIdx$ of the minimum of $a[i..n]$, and then $a[i]$ and $a[minIdx]$ are swapped (if $minIdx \neq i$)

Algorithm: SelectionSort (1/2)

Input : An array $a[1..n]$ of n natural numbers
Output : The array a sorted in non-decreasing order

```
1  function SelectionSort(a, n)
2      i := 1
3      while i ≤ n − 1 do
4          minIdx := i   // index of the minimum in a[i..n]
5          j := i + 1
6          while j ≤ n do
7              if a[j] < a[minIdx] then
8                  minIdx ← j          // new minimum found
9              end
10             j ← j + 1
11         end
```

Algorithm: SelectionSort (2/2)

```
function SelectionSort(a, n) (continued)
    while i ≤ n − 1 (continued) do
12      if minIdx ≠ i then
13          tmp := a[i]              // swap a[i] and a[minIdx]
14          a[i] ← a[minIdx]
15          a[minIdx] ← tmp
16      end
17      i ← i + 1
18  end
19 end
```

Run in Colab: Python · C++ · Java

How SelectionSort works

- The outer loop iterates with index i from 1 to $n - 1$
- For each i, the inner loop scans $a[i + 1..n]$ and keeps in *minIdx* the index of the smallest key met so far
- Since the comparison at line 7 is strict, in case of ties *minIdx* is the index of the *first* minimum
- After the inner loop, if $minIdx \neq i$, the minimum $a[minIdx]$ is swapped with $a[i]$: at most **one swap** per iteration of the outer loop, unlike Exchange Sort
- After iteration i of the outer loop, the sub-array $a[1..i]$ is sorted and contains the i smallest keys of the array

Example: Selection Sort on $[5, 2, 4, 1, 3]$ (1/3)

Input array: $a = [5, 2, 4, 1, 3]$, with $n = 5$. The outer loop executes the iterations $i = 1, 2, 3, 4$.

Detailed trace (inner loop), iteration $i = 1$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	–	5	2	4	1	3	initial state
1	–	1	5	2	4	1	3	$minIdx := 1$ ($a[1] = 5$)
	2	2	5	2	4	1	3	$a[2] = 2 < a[1] = 5$: $minIdx \leftarrow 2$
	3	2	5	2	4	1	3	$a[3] = 4 \geq a[2] = 2$: no update
	4	4	5	2	4	1	3	$a[4] = 1 < a[2] = 2$: $minIdx \leftarrow 4$
	5	4	5	2	4	1	3	$a[5] = 3 \geq a[4] = 1$: no update
	–	–	1	2	4	5	3	$minIdx = 4 \neq 1$: swap $a[1] \leftrightarrow a[4]$

Example: Selection Sort on $[5, 2, 4, 1, 3]$ (2/3)

Detailed trace (inner loop), iterations $i = 2$, $i = 3$ and $i = 4$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
2	–	2	1	2	4	5	3	$minIdx := 2$ ($a[2] = 2$)
	3	2	1	2	4	5	3	$a[3] = 4 \geq a[2] = 2$: no update
	4	2	1	2	4	5	3	$a[4] = 5 \geq a[2] = 2$: no update
	5	2	1	2	4	5	3	$a[5] = 3 \geq a[2] = 2$: no update
	–	–	1	2	4	5	3	$minIdx = 2$: no swap
3	–	3	1	2	4	5	3	$minIdx := 3$ ($a[3] = 4$)
	4	3	1	2	4	5	3	$a[4] = 5 \geq a[3] = 4$: no update
	5	5	1	2	4	5	3	$a[5] = 3 < a[3] = 4$: $minIdx \leftarrow 5$
	–	–	1	2	3	5	4	$minIdx = 5 \neq 3$: swap $a[3] \leftrightarrow a[5]$
4	–	4	1	2	3	5	4	$minIdx := 4$ ($a[4] = 5$)
	5	5	1	2	3	5	4	$a[5] = 4 < a[4] = 5$: $minIdx \leftarrow 5$
	–	–	1	2	3	4	5	$minIdx = 5 \neq 4$: swap $a[4] \leftrightarrow a[5]$

Example: Selection Sort on $[5, 2, 4, 1, 3]$ (3/3)

State of the array after each iteration of the outer loop:

i	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	5	2	4	1	3	initial state
1	4	1	2	4	5	3	swap $a[1] \leftrightarrow a[4]$: $a[1] = \min(a[1..5]) = 1$
2	2	1	2	4	5	3	no swap: $a[2] = \min(a[2..5]) = 2$
3	5	1	2	3	5	4	swap $a[3] \leftrightarrow a[5]$: $a[3] = \min(a[3..5]) = 3$
4	5	1	2	3	4	5	swap $a[4] \leftrightarrow a[5]$: $a[4] = \min(a[4..5]) = 4$

- The sorted array is $[1, 2, 3, 4, 5]$
- The algorithm performs 10 key comparisons and 3 swaps

Example: Selection Sort on $[12, 11, 13, 5, 6]$ (1/3)

Input array: $a = [12, 11, 13, 5, 6]$, with $n = 5$.

Detailed trace (inner loop), iterations $i = 1$ and $i = 2$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	–	12	11	13	5	6	initial state
1	–	1	12	11	13	5	6	$minIdx := 1$ ($a[1] = 12$)
	2	2	12	11	13	5	6	$a[2] = 11 < a[1] = 12$: $minIdx \leftarrow 2$
	3	2	12	11	13	5	6	$a[3] = 13 \geq a[2] = 11$: no update
	4	4	12	11	13	5	6	$a[4] = 5 < a[2] = 11$: $minIdx \leftarrow 4$
	5	4	12	11	13	5	6	$a[5] = 6 \geq a[4] = 5$: no update
	–	–	5	11	13	12	6	$minIdx = 4 \neq 1$: swap $a[1] \leftrightarrow a[4]$
2	–	2	5	11	13	12	6	$minIdx := 2$ ($a[2] = 11$)
	3	2	5	11	13	12	6	$a[3] = 13 \geq a[2] = 11$: no update
	4	2	5	11	13	12	6	$a[4] = 12 \geq a[2] = 11$: no update
	5	5	5	11	13	12	6	$a[5] = 6 < a[2] = 11$: $minIdx \leftarrow 5$
	–	–	5	6	13	12	11	$minIdx = 5 \neq 2$: swap $a[2] \leftrightarrow a[5]$

Example: Selection Sort on $[12, 11, 13, 5, 6]$ (2/3)

Detailed trace (inner loop), iterations $i = 3$ and $i = 4$:

i	j	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
3	–	3	5	6	13	12	11	$minIdx := 3$ ($a[3] = 13$)
	4	4	5	6	13	12	11	$a[4] = 12 < a[3] = 13$: $minIdx \leftarrow 4$
	5	5	5	6	13	12	11	$a[5] = 11 < a[4] = 12$: $minIdx \leftarrow 5$
	–	–	5	6	11	12	13	$minIdx = 5 \neq 3$: swap $a[3] \leftrightarrow a[5]$
4	–	4	5	6	11	12	13	$minIdx := 4$ ($a[4] = 12$)
	5	4	5	6	11	12	13	$a[5] = 13 \geq a[4] = 12$: no update
	–	–	5	6	11	12	13	$minIdx = 4$: no swap

Example: Selection Sort on $[12, 11, 13, 5, 6]$ (3/3)

State of the array after each iteration of the outer loop:

i	$minIdx$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	Action
–	–	12	11	13	5	6	initial state
1	4	5	11	13	12	6	swap $a[1] \leftrightarrow a[4]$: $a[1] = \min(a[1..5]) = 5$
2	5	5	6	13	12	11	swap $a[2] \leftrightarrow a[5]$: $a[2] = \min(a[2..5]) = 6$
3	5	5	6	11	12	13	swap $a[3] \leftrightarrow a[5]$: $a[3] = \min(a[3..5]) = 11$
4	4	5	6	11	12	13	no swap: $a[4] = \min(a[4..5]) = 12$

- The sorted array is $[5, 6, 11, 12, 13]$
- The algorithm performs 10 key comparisons and 3 swaps

We now count the comparisons and the swaps of Selection Sort,
and we check whether it is in-place and stable.

Observation 5: comparisons and swaps of Selection Sort

Given an array $a[1..n]$ *of* n *natural numbers,* `SelectionSort` *performs exactly* $\frac{n(n-1)}{2}$ *key comparisons and at most* $n-1$ *swaps. No swap is performed when the array is already sorted, and* $n-1$ *swaps are performed, for instance, on the array* $[2, 3, \ldots, n, 1]$.

Proof (1/2)

- As for Exchange Sort, in iteration $i \in \{1, 2, \ldots, n-1\}$ the comparison at line 7 is executed once for each $j \in \{i+1, i+2, \ldots, n\}$, so the total number of key comparisons is

$$\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$$

  by the sum of the first $n-1$ positive integers
- Each of the $n-1$ iterations of the outer loop performs at most one swap (lines 13–15), so there are at most $n-1$ swaps
- If the array is sorted, the minimum of $a[i..n]$ is $a[i]$ and, by the strict comparison at line 7, *minIdx* remains equal to i: no swap is performed

Proof (2/2)

- On $[2, 3, \ldots, n, 1]$, iteration 1 finds the minimum 1 in position n and swaps it with $a[1] = 2$, giving $[1, 3, 4, \ldots, n, 2]$
- In general, at the beginning of iteration i the array is

$$[1, 2, \ldots, i-1, i+1, \ldots, n, i],$$

  the minimum i of $a[i..n]$ is in position $n \neq i$, and the swap gives

$$[1, 2, \ldots, i, i+2, \ldots, n, i+1]$$

- Hence each of the $n-1$ iterations performs a swap

□

Few assignments, fixed comparisons

- Selection Sort performs at most $3(n-1)$ key assignments
- This is the smallest worst-case number of key assignments among the three algorithms of this chapter
- Its number of comparisons, $\frac{n(n-1)}{2}$, does not depend on the input

Observation 6: Selection Sort is in-place but not stable

`SelectionSort` is an in-place sorting algorithm, but it is not stable.

Proof

- Besides the array, the algorithm uses only the four variables i, j, *minIdx* and *tmp*: it is in-place
- Consider the array of records $[(2, A),\ (2, B),\ (1, C)]$
- For $i = 1$, the minimum is $(1, C)$, with $minIdx = 3$, and the swap of $a[1]$ and $a[3]$ gives $[(1, C),\ (2, B),\ (2, A)]$
- For $i = 2$, the comparison $2 < 2$ is false, so $minIdx = 2$ and no swap is performed
- In the output $(2, A)$ follows $(2, B)$, while it preceded it in the input: the swap moved $(2, A)$ past the equal key $(2, B)$



From pseudo-code to code

- In the implementations (C++, Python and Java, in the Colab notebooks above) the indices go from 0 to $n-1$
- The outer loop runs with i from 0 to $n-2$, and *minIdx* starts from i; the inner loop runs with j from $i+1$ to $n-1$

Summary

Selection Sort:

- In iteration i, selects the position $minIdx$ of the minimum of $a[i..n]$, then swaps $a[i]$ and $a[minIdx]$ (if $minIdx \neq i$)

Cost:

- Always exactly $\frac{n(n-1)}{2}$ key comparisons
- At most $n-1$ swaps, i.e., at most $3(n-1)$ key assignments

Properties:

- In-place, but not stable

Comparison of the three algorithms

Introduction: Comparison of the three algorithms

- We put side by side the results of the six Observations of the chapter
- We compare the three algorithms on the two arrays used throughout the chapter
- The comparison tells us which algorithm to choose depending on what matters: comparisons, moves of keys or stability

The three algorithms side by side

- The table collects the results of the six Observations of this chapter, for an array of n keys
- The "moves" are the shifts for Insertion Sort and the swaps for the other two algorithms

	key comparisons		moves			
algorithm	best	worst	best	worst	in-place	stable
Insertion Sort	$n-1$	$\frac{n(n-1)}{2}$	0 shifts	$\frac{n(n-1)}{2}$ shifts	yes	yes
Exchange Sort	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	0 swaps	$\frac{n(n-1)}{2}$ swaps	yes	no
Selection Sort	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	0 swaps	$n-1$ swaps	yes	no

Example

The three algorithms on the same arrays

On the two arrays used throughout the chapter, the three algorithms perform the following numbers of key comparisons and moves.

	$[5, 2, 4, 1, 3]$		$[12, 11, 13, 5, 6]$	
algorithm	comparisons	moves	comparisons	moves
Insertion Sort	9	7 shifts	9	7 shifts
Exchange Sort	10	7 swaps	10	7 swaps
Selection Sort	10	3 swaps	10	3 swaps

Which algorithm?

- Insertion Sort, Exchange Sort and Selection Sort are comparison-based and in-place, and in the worst case all three perform $\frac{n(n-1)}{2}$ key comparisons
- Insertion Sort adapts to the input: on an array that is already sorted it performs only $n-1$ comparisons and no shift. It is the only stable algorithm of the three
- Exchange Sort and Selection Sort always perform $\frac{n(n-1)}{2}$ comparisons, but Selection Sort performs at most $n-1$ swaps, against the $\frac{n(n-1)}{2}$ swaps of Exchange Sort in the worst case

Summary (1/2)

The sorting problem:

- Rearrange the keys of $a[1..n]$ in non-decreasing order
- Comparison-based algorithms only compare and copy keys; we count key comparisons and key assignments in the best and worst case, extra memory and stability

Insertion Sort:

- Inserts each key in the sorted prefix, shifting right the greater keys
- From $n-1$ to $\frac{n(n-1)}{2}$ comparisons, at most $\frac{n(n-1)}{2}$ shifts; in-place and stable

Summary (2/2)

Exchange Sort:

- Brings the minimum of $a[i..n]$ into position i with repeated swaps
- Exactly $\frac{n(n-1)}{2}$ comparisons, at most $\frac{n(n-1)}{2}$ swaps; in-place, not stable

Selection Sort:

- Selects the minimum of $a[i..n]$ and swaps it once into position i
- Exactly $\frac{n(n-1)}{2}$ comparisons, at most $n-1$ swaps; in-place, not stable

Choice: Insertion Sort adapts to the input and is stable; Selection Sort moves the keys the least